

Feature Scaling & Normalization

. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain why feature scaling is necessary and identify which machine learning algorithms require it
- Apply MinMax scaling and standard scaling using scikit-learn, and choose the right technique for your data
- Combine a train-test split with scaling in the correct order to prevent data leakage
- Handle datasets with outliers using a robust scaler

2. Detailed Explanation

Features and why scaling matters

A **feature** is any input variable that helps a model make predictions. Features are also called **independent variables**. The variable being predicted is the **dependent variable**, or **target variable**.

For example, in a house price prediction task the features are square feet area, locality, number of BHK, and amenities — and the target is the selling price. In a student marks prediction task the features are previous exam marks, hours studied, and attendance — and the target is the marks scored.

Now here is the problem. Features often live on wildly different numerical scales. Imagine a dataset with an age column (roughly 1–100) alongside a salary column (roughly 10,000–10,000,000). Any model that uses distance or magnitude in its decision-making will automatically assign far more weight to salary — simply because its numbers are larger. The age signal gets drowned out entirely.

Feature scaling solves this by bringing every feature onto a comparable scale before the model sees the data.

Algorithms that need scaling — and those that don't

Distance-based algorithms (KNN, K-Means, SVM) measure how far apart data points are, so the raw magnitude of each feature directly affects results. Optimization-based and variance-based algorithms are also sensitive to feature magnitude. All of the following require scaled features:

Algorithm	Why scaling matters
K-Nearest Neighbors (KNN)	Uses Euclidean distance to find the nearest neighbors
K-Means	Assigns points to cluster centroids based on distance
Support Vector Machine (SVM)	Defines margin boundaries using support vectors and distance
Logistic Regression	Gradient-based optimization converges faster with scaled inputs
Linear Regression	Also impacted, though the effect is smaller than the above
Principal Component Analysis (PCA)	Uses covariance and variance calculations that are sensitive to feature magnitude

Tree-based algorithms work differently. They make decisions by checking threshold conditions at each node — essentially a series of "if-else" splits. When you rescale a feature, its values change, but the relative order of values stays the same. So every split condition and every resulting branch remains identical.

Consider a split condition of "salary > 250,000". After rescaling salary to a 1–5 range, that same split becomes "salary > 2.5". Every data point still takes the same branch. For this reason, **tree-based algorithms do not require feature scaling**:

- **Decision Trees**
- **Random Forests**
- **XGBoost**

MinMax scaling — compressing data into [0, 1]

MinMax scaling is a technique that transforms every value in a column so that it falls between 0 and 1. This category of transformation is called **normalization**.

The formula is:

$$X_{\text{scaled}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

The minimum value in the column maps to 0 and the maximum maps to 1. Every other value falls proportionally between them. Crucially, MinMax scaling never produces negative values.

Here is how to apply it with scikit-learn:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
df = pd.DataFrame({"marks": [40, 60, 50]})
mn = MinMaxScaler()
min_max_marks = mn.fit_transform(df[["marks"]])
print(min_max_marks)
```

The `fit_transform` call learns the minimum and maximum from the data and applies the transformation in one step. The double brackets around 'marks' — `df[["marks"]]` — are required because the scaler expects a two-dimensional input (a DataFrame, not a plain Series).

When to reach for MinMax scaling:

- Your algorithm is KNN, K-Means, or a neural network
- Your data has known, meaningful natural limits — pixel values always fall in 0–255, probability values in 0–1, percentages in 0–100
- Your data is not normally distributed — MinMax scaling preserves the original shape of the distribution rather than forcing it into a bell curve

Watch out for: MinMax scaling is very sensitive to outliers. If one value is extremely large, it becomes the maximum in the denominator and compresses all other values into a tiny range near zero. More on this in the Robust Scaler section below.

Standard scaling — centering data around zero

Standard scaling (also called **standardization**) transforms data so it has a mean of 0 and a standard deviation of 1 — the shape of a standard normal (Gaussian) distribution.

The formula is:

$$X_{\text{scaled}} = (X - \text{mean}) / \text{standard_deviation}$$

Unlike MinMax scaling, the output is not bounded between 0 and 1. Standard scaling can produce negative values, and there is no fixed upper or lower limit.

```
from sklearn.preprocessing import StandardScaler
df = pd.DataFrame({"marks": [40, 60, 50]})
sc = StandardScaler()
standard_scale_marks = sc.fit_transform(df[["marks"]])
print(standard_scale_marks)
```

When to reach for standard scaling:

- Your data already follows a normal (bell-curve / Gaussian) distribution
- Your algorithm is Linear Regression, Logistic Regression, SVM, or PCA — these are conventionally paired with standardization

Choosing between MinMax and standard scaling

Here is a side-by-side comparison to help you decide:

Property	MinMax Scaling (Normalization)	Standard Scaling (Standardization)
Output range	0 to 1 (fixed)	Unbounded; centered at 0
Negative values possible	No	Yes
Best when data follows	Any distribution	Normal (Gaussian) distribution
Preferred algorithms	KNN, K-Means, neural networks	Linear Regression, Logistic Regression, SVM, PCA
Sensitivity to outliers	Severe — compresses all other values	Severe — outliers shift the mean

Neither technique is universally better. The data distribution and the algorithm guide the choice. Use the decision flow below as a quick guide:

Scaling multiple columns in practice

In real datasets you rarely have just one numeric column. Scaling each column individually is tedious and easy to get wrong. The standard approach is to apply the scaler to the entire DataFrame at once, then reconstruct the DataFrame with the original column names.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

```
df = pd.DataFrame({
    "age": [20, 30, 40],
    "salary": [30000, 50000, 100000]
})
sc = StandardScaler()
scaled = sc.fit_transform(df)
scaled_df = pd.DataFrame(scaled, columns = df.columns)

mn = MinMaxScaler()
min_scaled = mn.fit_transform(df)
mn_scaled_df = pd.DataFrame(mn_scaled, columns = df.columns)
```

Passing `df.columns` when constructing the new `DataFrame` preserves the original column names. The values have been rescaled for modeling, but the meaning of each column stays clear.

Train-test split

Before training any model, you divide the dataset into two parts: a **training set** (the model learns from this) and a **test set** (used to evaluate how well the model generalizes to data it has never seen).

```
from sklearn.model_selection import train_test_split

# df is a loan approval dataset with columns: age, salary & approval
X = df[["age", "salary"]] → Feature DataFrame - Independent Variables
y = df[["approval"]] → Target Series - Dependent Variables
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size = 0.2,
    random_state = 45
)
```

Key parameters:

- **test_size=0.2** — allocates 20% of rows to the test set; the remaining 80% becomes the training set automatically
- **random_state** — an integer seed that makes the split reproducible; any fixed integer (e.g., 42, 45) works as long as you use the same value every run

`train_test_split` always returns values in a fixed order: `X_train`, `X_test`, `y_train`, `y_test`. Unpacking in any other order will silently mix up your data.

Data leakage and the correct order of operations

Data leakage occurs when information from the test set influences the training process. The test set is meant to simulate unseen, real-world data. If it leaks into training, the model becomes overly optimistic and fails to generalize.

Scaling is a surprisingly common source of leakage. Here is how it happens:

Incorrect approach — scale the entire dataset first, then split: When you fit a `StandardScaler` on the full dataset, it computes the mean and standard deviation using every row — including future test rows. Those test-set values have now influenced the scaling parameters used during training. The model has indirectly "seen" the test data.

Correct approach — split first, then scale:

```
from sklearn.preprocessing import StandardScaler
```

```
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train) → Learn parameters from learning data only
```

```
X_test = sc.transform(X_test) → apply those same parameters to test data
```

This brings us to the distinction between `fit_transform` and `transform`:

Method	What it does	When to use
<code>fit_transform(data)</code>	Learns scaling parameters from data AND applies the transformation	Training data only
<code>transform(data)</code>	Applies already-learned parameters without re-learning	Test data and any new unseen data

Using `fit_transform` on the test set would re-learn new parameters from test-only statistics, producing an inconsistent scale and leaking information at the same time.

Robust scaler — scaling data with outliers

Both `MinMax` and standard scaling break down when your data contains extreme values.

With **MinMax scaling**, the outlier becomes the maximum in the denominator. If your values are 1, 2, 3, 4, and 10,000, the denominator is approximately 10,000. The scaled values for 1 through 4 end up less than 0.001 — their spread is completely lost.

With **standard scaling**, the mean is heavily distorted by extreme values. Because standardization subtracts the mean, a heavily skewed mean throws off the entire transformation.

Imagine a salary dataset where most employees earn 10,000–42,000 per month but one person earns 600,000. Both scalers produce poor results.

Robust scaler solves this by replacing the outlier-sensitive statistics:

- Instead of **mean** → uses **median** (the midpoint of the sorted data; not pulled by extreme values)
- Instead of **standard deviation** → uses **interquartile range (IQR)** (the spread between the 25th and 75th percentiles; outliers lie outside this range and do not distort it)

```
from sklearn.preprocessing import RobustScaler
```

```
salary_data = [[10000], [20000], [35000], [42000], [600000]]
```

```
scaler = RobustScaler()
```

```
r_scaled = scaler.fit_transform(salary_data)
```

```
print(r_scaled)
```

```
# Example Output: approximately [-1.14], [-0.68], [0.], [0.32], [25.68]
```

The output is not bounded between 0 and 1, and the outlier (600,000) still gets a large scaled value. But the non-outlier values maintain a meaningful spread relative to each other — which is exactly the goal. Whenever outlier detection reveals significant extreme values in your data, prefer robust scaler over MinMax or standard scaler.

3. Key Takeaways

- Feature scaling is essential for distance-based algorithms (KNN, K-Means, SVM, Logistic Regression, PCA) but not for tree-based algorithms (Decision Trees, Random Forests, XGBoost), because tree splits depend on value ordering, not magnitude.
- **MinMax scaling** squeezes data into [0, 1] — best for KNN, K-Means, neural networks, and data with fixed natural bounds like pixel values (0–255) or probabilities (0–1).
- **Standard scaling** centers data at mean 0 with standard deviation 1 — best for normally distributed data and linear models including Linear Regression, Logistic Regression, SVM, and PCA.
- Always split your data first and then scale: use `fit_transform` on the training set to learn parameters, and `transform` (not `fit_transform`) on the test set. Fitting on the full dataset before splitting causes data leakage.

- When your data contains significant outliers, use **Robust Scaler**, which relies on the median and IQR instead of the mean and standard deviation — making it immune to extreme values.

Mental model: Think of feature scaling as converting measurements to the same unit system before comparing them — just as you would not compare distances in miles alongside distances in millimeters without converting first.